

Análise e Implementação de Estruturas de Grafos

Eduardo Henrique Moreira dos Santos¹, João Guilherme Falante Araújo¹,
Mariana Almeida Henriques¹, Roberta Sophia Carvalho Silva¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais
Belo Horizonte – MG – Brasil

Abstract. *This work is divided into two main stages. The first stage involves the development of a custom library for graph manipulation and analysis, implementing fundamental data structures such as adjacency lists and adjacency matrices. This step was crucial to deepen the understanding of core concepts and establish a solid foundation for the subsequent phase. In the second stage, data mining was performed using the GitHub API on the NumPy repository [Oliphant et al. 2025], a key library in the Python scientific ecosystem. From the collected repository data, a graph was constructed. Various algorithms were applied to this graph to detect influential nodes, identify communities, and analyze subgraph densities. The results were then visualized using the Gephi platform.*

Resumo. *Este trabalho está dividido em duas etapas principais. A primeira etapa envolve o desenvolvimento de uma biblioteca personalizada para manipulação e análise de gráficos, implementando estruturas de dados fundamentais, como listas de adjacência e matrizes de adjacência. Essa etapa foi crucial para aprofundar a compreensão dos conceitos principais e estabelecer uma base sólida para a fase subsequente. Na segunda etapa, a mineração de dados foi realizada usando a API do GitHub no repositório NumPy [Oliphant et al. 2025], uma biblioteca importante no ecossistema científico do Python. A partir dos dados coletados do repositório, foi construído um grafo de interações. Vários algoritmos foram aplicados a esse gráfico para detectar nós influentes, identificar comunidades e analisar densidades de subgráficos. Os resultados foram então visualizados usando a plataforma Gephi.*

1. Introdução

Este trabalho prático da disciplina de Teoria dos Grafos tem como objetivo aplicar tanto os fundamentos teóricos quanto os aspectos práticos da área, por meio da criação de uma biblioteca de grafos e da análise de dados reais extraídos da API do Github.

Este relatório documenta todas as etapas do trabalho, desde a concepção e implementação da biblioteca até a extração, modelagem, análise e visualização dos dados, com foco na aplicação prática dos conceitos aprendidos em sala de aula.

2. Metodologia

O projeto está dividido em duas etapas principais. A primeira consiste no desenvolvimento de uma biblioteca própria para manipulação e análise de grafos, implementando estruturas básicas de lista de adjacência e matriz de adjacência. Essa etapa foi essencial

para consolidar o entendimento dos conceitos fundamentais e estabelecer uma base sólida para a etapa seguinte.

Na segunda parte do trabalho, foi realizada a mineração de dados por meio da API do Github no repositório NumPy [Oliphant et al. 2025]., que é uma das bibliotecas mais fundamentais do ecossistema Python científico. A partir dos dados coletados do repositório foi construído um grafo. Sobre esse grafo foram aplicados algoritmos para detectar nós influentes, identificar comunidades e densidades dos sub-grafos. Além disso, os resultados foram visualizados na plataforma Gephi. A metodologia está resumida na Figura 1.

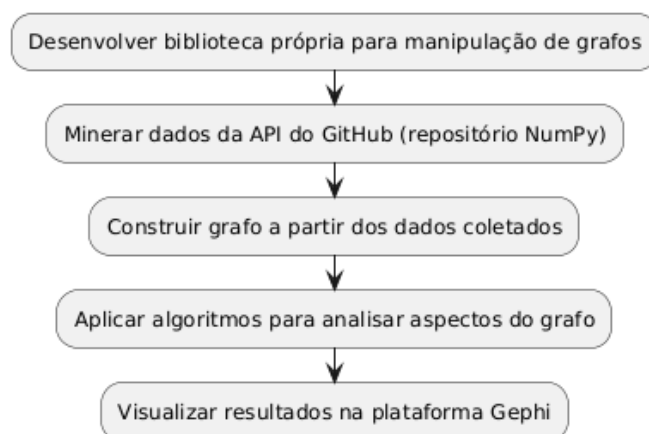


Figura 1. Etapas da metodologia adotada

3. Resultados

3.1. Biblioteca de grafos

O objetivo desta etapa foi criar uma biblioteca de grafos com suporte a operações fundamentais e algoritmos clássicos. Para fazer isso, foi selecionada pelo grupo a linguagem Python. A biblioteca foi desenvolvida por todos os integrantes do grupo no repositório graphs-project [Henriques 2025], no Github.

A base da implementação está nas classes `MatrixGraph` e `ListGraph` que estendem a classe `Graph` e utilizam listas de objetos `Vertex` e `Edge` para representar vértices e arestas, além da estrutura de matriz para armazenar a conectividade e pesos entre os nós (Figura 2).

Entre as principais funcionalidades implementadas destacam-se:

1. Criação de vértices e arestas, com suporte à inserção manual ou via métodos de biblioteca (`lib_add_vertex`, `lib_add_edge`).
2. Atribuição de pesos e rótulos tanto para vértices quanto para arestas, oferecendo uma representação mais rica dos elementos do grafo.
3. Remoção de arestas a partir de rótulos ou identificação pelos vértices de origem e destino.
4. Visualização da matriz de adjacência, para inspeção direta da estrutura de conectividade.

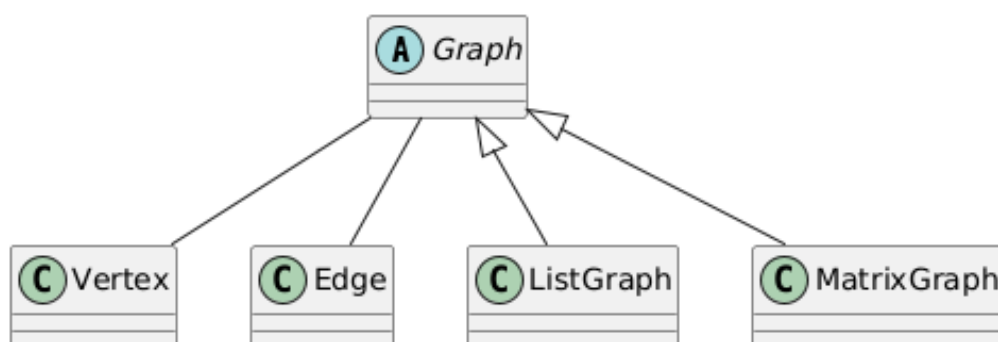


Figura 2. Classes construídas na biblioteca

Além disso, foram implementadas verificações estruturais, como:

1. Adjacência entre vértices e entre arestas,
2. Incidência entre vértices e arestas,
3. Existência de arestas,
4. Contagem de vértices e arestas,
5. Determinação se o grafo é vazio ou completo.

Além da versão interativa (voltada para aprendizado e testes em tempo real), foi desenvolvida uma interface de biblioteca com métodos reutilizáveis, permitindo a integração da estrutura em aplicações maiores ou scripts automatizados.

3.2. Mineração de dados

3.2.1. Fonte de dados e API utilizada

Para a coleta de dados deste trabalho, utilizou-se a API GraphQL v4 do GitHub, acessada através do endpoint <https://api.github.com/graphql>. Conforme mencionado anteriormente, o repositório selecionado para análise foi o NumPy, um dos projetos de código aberto mais relevantes no ecossistema Python para computação científica. A autenticação foi realizada mediante Personal Access Tokens (PAT) do GitHub, configurados com as permissões `public_repo` e `read:user`, garantindo acesso completo aos dados públicos do repositório. Para otimizar o processo e evitar limitações de taxa de requisições, foram utilizados quatro tokens distintos com rotação automática.

3.2.2. Estrutura dos dados extraídos

O processo de mineração focou na extração de quatro categorias principais de dados, conforme apresentado na Tabela 1.

Cada categoria de dados foi estruturada para preservar as relações hierárquicas e temporais, permitindo análises longitudinais e de rede social. Os campos de data seguem o padrão ISO 8601, garantindo consistência temporal para análises cronológicas.

Tabela 1. Estrutura dos dados minerados

Categoria	Campos Principais	Relacionamentos
Issues	ID, número, título, descrição, estado, datas, autor, <i>assignees</i> , <i>labels</i> , contadores	Base para comentários
Pull Requests	ID, número, título, descrição, estado, datas, autor, <i>assignees</i> , <i>labels</i> , métricas de código, contadores	Base para <i>reviews</i>
Comments	ID, conteúdo, datas, autor, reações	Vinculado à <i>issues</i>
Reviews	ID, conteúdo, estado, datas, autor, comentários	Vinculado a <i>pull requests</i>

3.2.3. Ferramentas e tecnologias empregadas

A implementação da mineração foi desenvolvida em Python 3.13, utilizando as seguintes bibliotecas especializadas:

- **requests**: Para execução de requisições HTTP/GraphQL ao endpoint da API;
- **pandas**: Para manipulação, transformação e análise dos dados coletados;
- **python-dotenv**: Para gerenciamento seguro de credenciais e variáveis de ambiente;
- **logging**: Para registro detalhado do processo de mineração e *debugging*;
- **time**: Para implementação de controles de taxa de requisições.

O código foi estruturado em módulos especializados: `github_miner.py` (classe principal), `main.py` (execução completa) e `quick_start.py` (testes).

3.2.4. Resultados da mineração

O processo de coleta resultou em um dataset abrangente, conforme detalhado na Tabela 2.

Tabela 2. Quantidades e características dos dados minerados

Tipo de Dado	Quantidade	Período Temporal	Tamanho (MB)
Issues	13.388	2008–2025	12,1
Pull Requests	15.803	2010–2025	10,5
Comments	49.532	2008–2025	22,2
Reviews	40.335	2011–2025	7,2
Total	119.058	17 anos	73,1

A coleta abrangeu um período de 17 anos (2008-2025), totalizando aproximadamente 5.000 requisições GraphQL executadas em cerca de 2 horas. A cobertura temporal completa permite análises longitudinais robustas sobre a evolução do projeto e padrões de colaboração. Os dados foram exportados em formato CSV com codificação UTF-8, estrutura tabular normalizada e campos de data padronizados, facilitando análises posteriores e garantindo interoperabilidade com ferramentas de análise de dados.

3.3. Construção do grafo

O grafo foi construído com base nos dados obtidos por meio da API do GitHub, durante a etapa de mineração, e utilizando a biblioteca desenvolvida na primeira parte do projeto. Após a coleta, os dados foram organizados em quatro arquivos CSV, cada um contendo registros distintos de interações entre usuários no repositório analisado:

1. Issues (abertura e fechamento)
2. Comentários
3. Pull requests (submissão e merge)
4. Revisões

A modelagem do grafo é resumida na Figura 3.

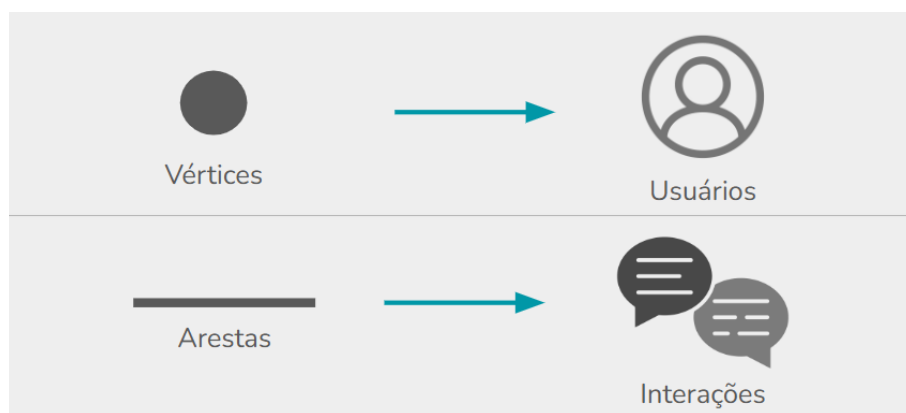


Figura 3. Modelagem do grafo

Os vértices representam os usuários. As arestas representam as interações entre esses usuários, sendo direcionadas — ou seja, possuem origem (quem iniciou a ação) e destino (quem foi o alvo da ação).

As arestas são ponderadas, com pesos atribuídos de acordo com o tipo de interação, conforme abaixo:

- A abre uma issue e B fecha: peso 3
- A abre um pull request e B faz merge ou fecha: peso 3
- B aprova ou solicita mudanças em um pull request de A: peso 2
- B comenta em uma issue ou pull request de A: peso 2

Antes da construção do grafo, os dados passaram por um processo de limpeza e pré-processamento, que incluiu a remoção de registros duplicados, normalização de identificadores de usuários e filtragem de interações inválidas ou incompletas.

3.4. Visualização e análise de resultados

A visualização inicial do grafo foi realizada utilizando a plataforma Gephi. O Gephi é uma plataforma de software de código aberto voltada para a visualização e análise de redes e grafos. Bastante utilizada nas áreas de ciência de dados, sociologia, biologia, marketing e ciência da computação, o Gephi permite explorar estruturas complexas de redes de forma interativa e intuitiva.

O uso dessa plataforma permite uma análise exploratória da rede bruta de interações e oferecendo insights iniciais sobre a estrutura da colaboração entre os usuários. O grafo construído está apresentado na Figura 4.

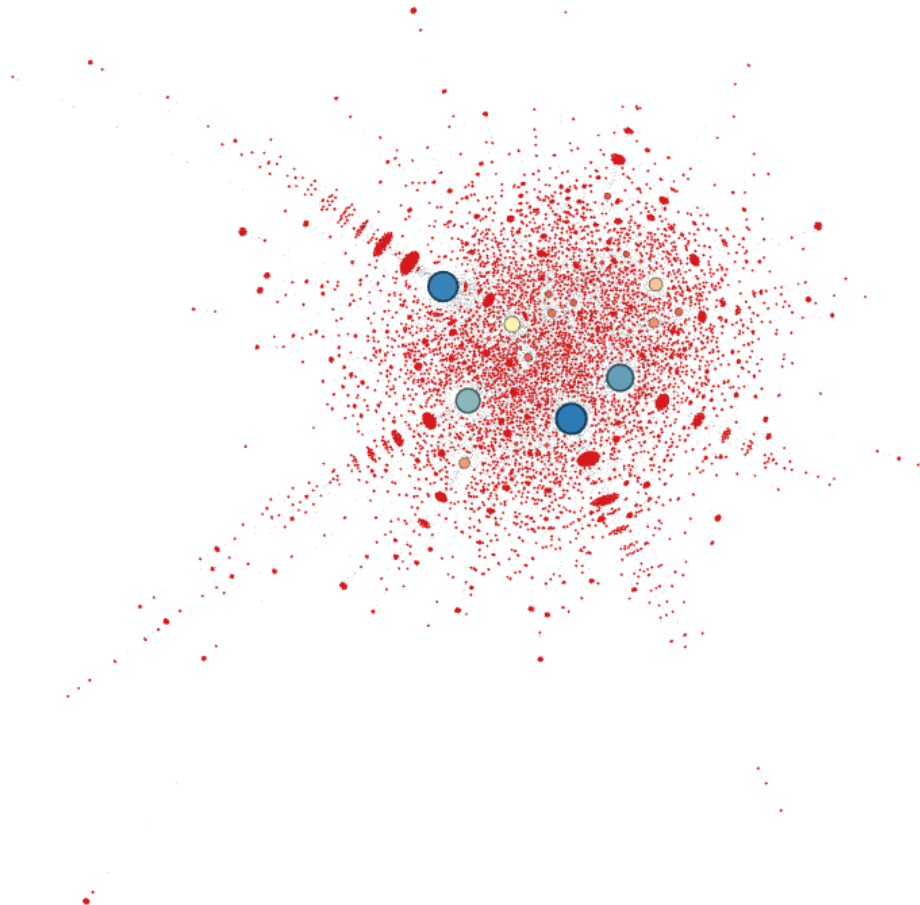


Figura 4. Visualização do grafo no Gephi

Além da visualização, foram realizadas análises sobre o grafo construído. Essas análises incluem a importação da biblioteca de grafos construída na primeira etapa e foram realizadas no repositório social-graph-mining [Silva 2025]. Para isso, foram avaliados seis aspectos, apresentados a seguir:

- Quem são os 5 usuários mais influentes?
- Quem gera maior fragmentação?
- Quais grupos naturais existem?
- Qual o nível de conexão da comunidade?
- Dado um usuário, quem são os mais próximos?
- Dado um usuário, quem são os mais próximos que não interagem?

Para realizar essas avaliações, foram construídas funções de manipulação do grafo, e o detalhamento de cada uma das avaliações é apresentado a seguir.

3.4.1. Usuários mais influentes

Para descobrir os usuários mais influentes, foi implementado o método `calculate_weighted_degrees()`, que calcula a influência de cada usuário somando o grau de entrada ponderado (interações recebidas, como comentários e revisões em seus *pull requests* ou *issues*) com o grau de saída ponderado (interações iniciadas, como comentários e revisões feitas por ele). Essa métrica reflete tanto a participação ativa do usuário quanto sua relevância no projeto, permitindo ranqueá-los com base no peso total de suas interações na rede.

A Figura 5 apresenta o trecho de código responsável pelo cálculo da influência dos usuários. Já a Figura 6 exibe a visualização gerada no Gephi, destacando os cinco usuários mais influentes identificados com base na métrica proposta.

```
171     def calculate_weighted_degrees(self):
172         """Calcula o grau ponderado de entrada + saída de cada vértice"""
173         weighted_degrees = {}
174         ids = [v.id for v in self.graph.vertices]
175         n = len(ids)
176
177         for i in range(n):
178             vertex_id = ids[i]
179             out_degree = sum(self.graph.matrix[i][j] for j in range(n))
180             in_degree = sum(self.graph.matrix[j][i] for j in range(n))
181             weighted_degrees[vertex_id] = out_degree + in_degree
182
183         return weighted_degrees
184
```

Figura 5. Código para calcular os usuários mais influentes

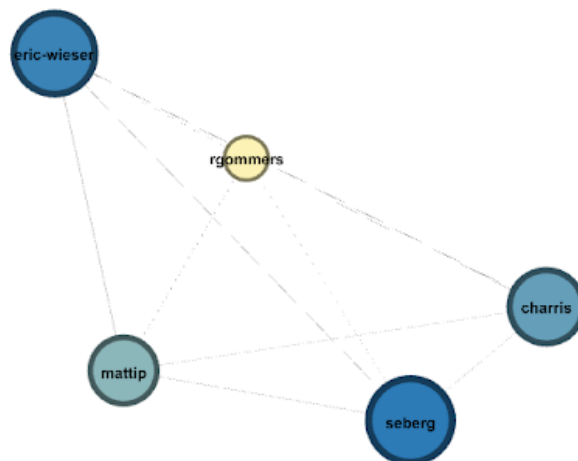


Figura 6. Visualização no Gephi dos usuários mais influentes

3.4.2. Remoção de quem gera maior fragmentação

Para descobrir os usuários que podem causar a maior fragmentação, foi implementado o método `identify_influential_users()`, que chama o método

`calculate_user_scores()` e identifica os usuários que mais contribuem para a fragmentação da rede ao analisar seu grau total de conexões (entrada + saída, sem pesos). O foco está em colaboradores com muitas interações dispersas que criam subgrupos isolados em vez de fortalecer a coesão da comunidade, permitindo avaliar o impacto de sua possível remoção para reduzir a desconexão na rede.

Na Figura 7, é possível observar o código utilizado para identificar os usuários com maior potencial de fragmentação. A Figura 8 mostra o efeito visual da remoção desses usuários no Gephi, evidenciando a redução da desconexão na rede.

```
154 def calculate_user_scores(self):
155     """Calcula o grau total (entrada + saída) de cada usuário, desconsiderando pesos"""
156     user_scores = defaultdict(int)
157     ids = [v.id for v in self.graph.vertices]
158     n = len(ids)
159
160     for i in range(n):
161         vertex_id = ids[i]
162         out_degree = sum(1 for j in range(n) if self.graph.matrix[i][j] > 0)
163         in_degree = sum(1 for j in range(n) if self.graph.matrix[j][i] > 0)
164         user_scores[vertex_id] = in_degree + out_degree
165
166     return user_scores
167
```

Figura 7. Método para identificar qual usuário gera maior fragmentação

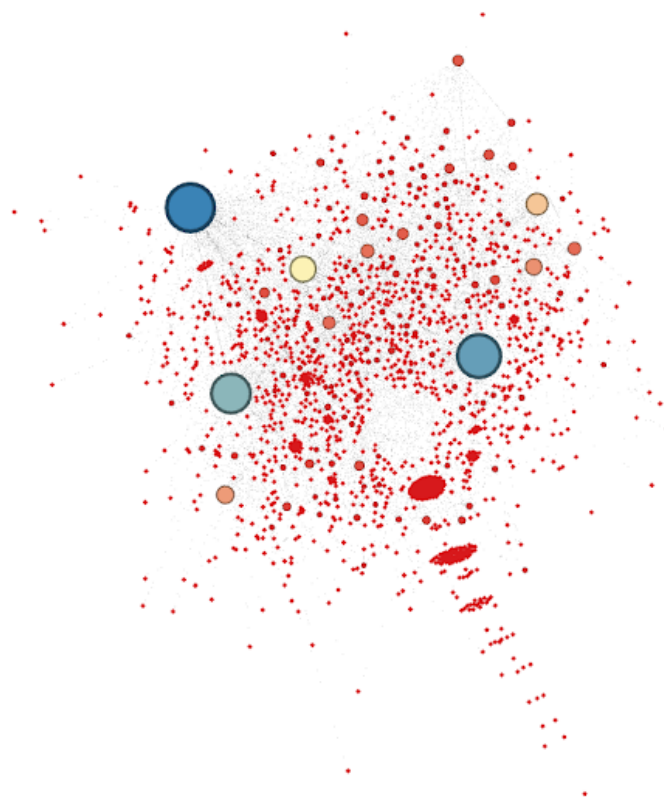


Figura 8. Visualização no Gephi após retirada do usuário que gera maior fragmentação

3.4.3. Grupos naturais existentes

Utilizando o algoritmo, foram identificados 5.963 grupos naturais distintos no grafo construído a partir de dados minerados do GitHub. Os dados indicam que a maioria desses grupos possui apenas um membro, o que é consistente com o comportamento de contas isoladas ou com baixa atividade colaborativa. No entanto, também foram detectados agrupamentos expressivos.

A maior comunidade identificada conta com 621 usuários conectados, incluindo contribuintes notórios como `eric-wieser`, `seberg`, `charris`, `mattipe` e `rgommers`. Essa comunidade apresentou um nível de conexão interna de apenas 1,15%, o que revela que, apesar do grande número de membros, existem muitos vínculos unidirecionais ou interações esparsas. Esse fenômeno também reforça a importância da presença de usuários altamente conectados que atuam como pontes dentro da comunidade.

As demais comunidades com apenas um membro apresentaram conexão interna de 100%, o que é esperado, já que esses nós formam grupos isolados. A análise dos grupos naturais, portanto, permitiu evidenciar a existência de núcleos de colaboração intensa, bem como de zonas periféricas.

3.4.4. Nível de conexão da comunidade

A identificação de comunidades em grafos é fundamental para compreender a estrutura e a dinâmica de redes complexas, uma vez que comunidades representam grupos de nós com alta densidade de conexões internas e menor conexão com o restante do grafo. A análise dessas comunidades permite revelar padrões relevantes, como agrupamentos naturais, compartilhamento de interesses, ou influência coletiva dentro do sistema representado.

As comunidades analisadas foram selecionadas a partir dos top 5 grupos naturais identificados no grafo, correspondendo às partições com maior coesão interna segundo o algoritmo de detecção utilizado. Essa escolha prioriza os conjuntos de nós que formam subestruturas densamente conectadas, refletindo agrupamentos significativos dentro da rede. Focar nesses grupos permite uma análise mais representativa e eficaz das dinâmicas de influência e interação presentes no grafo, destacando as regiões com maior relevância estrutural para o sistema estudado.

No contexto deste trabalho, o nível de conexão ou influência de uma comunidade foi mensurado pela soma dos graus totais (grau de entrada e saída) de todos os seus membros. Esta métrica reflete o volume agregado de interações que os nós da comunidade possuem, servindo como um indicativo da importância e do impacto coletivo daquele grupo dentro do grafo. A função `get_community_influence_by_degree` implementa esse cálculo, obtendo previamente os graus individuais de cada nó através do método `calculate_user_scores()` e somando-os para todos os membros da comunidade. Assim, comunidades com maior soma dos graus totais são consideradas mais influentes no contexto analisado, destacando-se pela sua centralidade e capacidade de propagar informações ou exercer influência na rede. O resultado é apresentado na Tabela 3. a seguir.

Tabela 3. Top 5 comunidades e nível de conexão

Posição	Descrição	Influência Total (interações)
1	Grupo com 621 membros	18.647
2	Grupo com 1 membro	2
3	Grupo com 1 membro	1
4	Grupo com 1 membro	1
5	Grupo com 1 membro	1

3.4.5. Usuários mais próximos dado um usuário

Dado um usuário, considera-se que os usuários mais próximos são aqueles que são diretamente conectados, isto é, vizinhos imediatos. A extração dessa informação foi realizada através do método `get_total_neighbors`, que recebe como parâmetro um nó (um usuário) e retorna a quantidade de número de vizinhos diretos deste usuário.

Obteve-se, para os cinco maiores usuários do grafo, o número de vizinhos diretos, que está apresentado na Tabela 4.

Tabela 4. Top 5 usuários do grafo e seus vizinhos diretos

Posição	Usuário	Nº de vizinhos diretos
1	seberg	1659
2	eric-wieser	1573
3	charris	1411
4	mattip	1267
5	rgommers	757

Além disso, essa informação pode ser analisada também de forma visual pelo Gephi. Para exemplificar os usuários mais próximos dado um usuário, foi selecionado o nó com ID “seberg” (Figura 9).

Para obter os usuários mais próximos, utilizou-se o filtro Ego Network, com o ID do nó e a profundidade 1, indicando que serão obtidos apenas os vizinhos imediatos a este nó. Realizando este processo, foi possível obter todos os vizinhos imediatos do nó com ID “seberg”, conforme Figura 10.

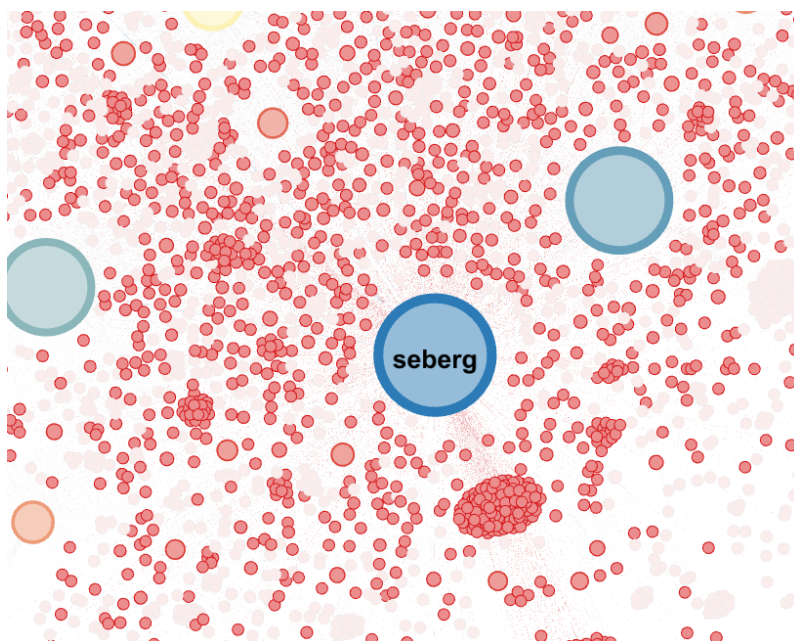


Figura 9. Usuário inicial para verificar usuários mais próximos

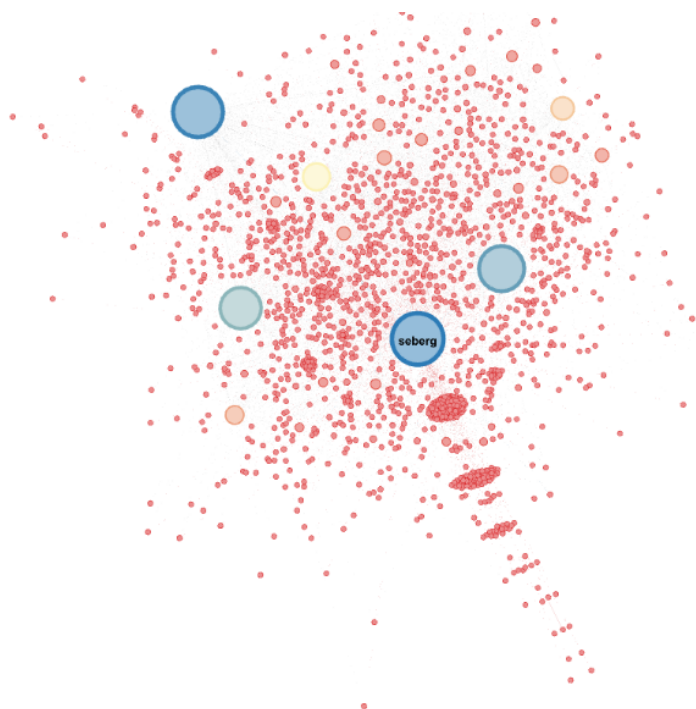


Figura 10. Usuários mais próximos dado o usuário com ID “seberg”

3.4.6. Usuários mais próximos que não interagem dado um usuário

Dado um usuário, considera-se que os usuários mais próximos que não interagem são aqueles que possuem um caminho curto até o nó (nós de distância 2 e 3, por exemplo), mas que não são vizinhos imediatos (conexões de grau 1). A extração dessa informação foi

realizada através do método `get_total_indirect_neighbors`, que recebe como parâmetro um nó (um usuário) e retorna a quantidade de número de vizinhos indiretos deste usuário.

Obteve-se, para os cinco maiores usuários do grafo, o número de vizinhos indiretos, que está apresentado na Tabela 5.

Tabela 5. Usuários mais próximos que não interagem diretamente dado um usuário

Posição	Usuário	Nº de usuários próximos sem interação direta
1	seberg	4885
2	eric-wieser	4968
3	charris	5131
4	mattip	5273
5	rgommers	5782

Além disso, essa informação pode ser analisada também de forma visual pelo Gephi. Para exemplificar os usuários mais próximos que não interagem dado um usuário, foi selecionado o nó com ID “mattip” (Figura 11).

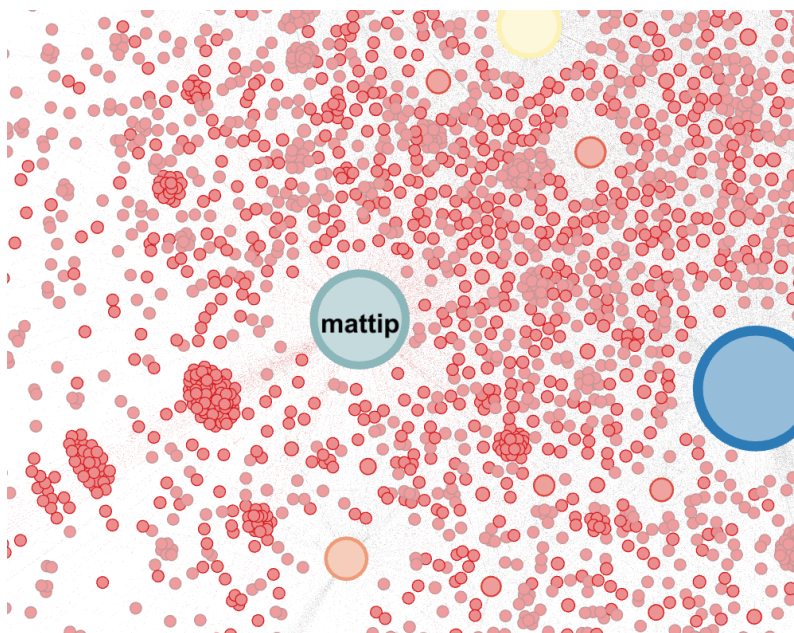


Figura 11. Usuário inicial para verificar usuários mais próximos que não interagem

Para obter os usuários mais próximos, utilizou-se o filtro Ego Network, com o ID do nó e a profundidade 2, indicando que serão obtidos apenas os usuários com conexão de grau 2 em relação ao nó selecionado. Realizando este processo, foi possível obter os usuários próximos mas que não interagem com o nó com ID “mattip”, conforme Figura 12.

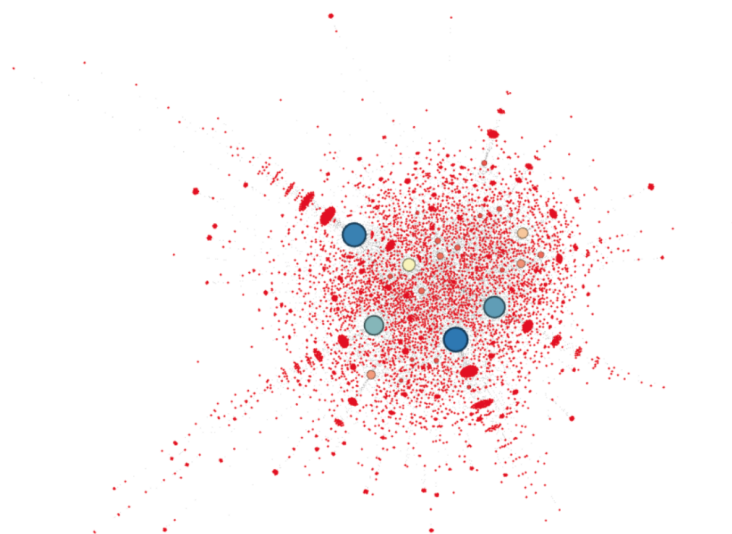


Figura 12. Usuários mais próximos mas que não interagem com o usuário com ID “mattip”

4. Conclusão

O desenvolvimento deste trabalho prático permitiu consolidar, de forma integrada, os conceitos fundamentais da teoria dos grafos e sua aplicação prática em problemas reais de análise de redes. A construção de uma biblioteca própria de grafos foi essencial para aprofundar o conhecimento sobre estruturas de dados, algoritmos clássicos e boas práticas de programação modular e reutilizável.

Na segunda etapa, a mineração de dados por meio de uma API e a modelagem de uma rede de interações representaram um desafio multidisciplinar, envolvendo desde o pré-processamento dos dados até a análise de estruturas complexas. A utilização do Gephi para visualizar comunidades, medir densidade dos sub-grafos, filtragem de nós vizinhos e outros permitiu extrair insights relevantes sobre a dinâmica da rede, além de exercitar a criatividade na formulação de métricas e estratégias analíticas.

A visualização e interpretação dos resultados contribuíram para tornar o grafo não apenas uma estrutura abstrata, mas uma ferramenta narrativa poderosa capaz de revelar padrões, destacar atores centrais e descrever o comportamento coletivo dos usuários.

Como possibilidades futuras, destaca-se a incorporação de dados temporais para análise de evolução da rede, aplicação de técnicas de aprendizado de máquina sobre o grafo e otimização dos algoritmos implementados para escalabilidade. Assim, o projeto se mantém aberto a extensões e aprofundamentos que podem enriquecer ainda mais o aprendizado e a aplicação da disciplina.

Referências

Henriques, M. A. (2025). graphs-project. <https://github.com/mmhqs/graphs-project/>. Acesso em: junho de 2025.

Oliphant, T. E., Harris, C. R., Virtanen, P., and outros (2025). Numpy: biblioteca fundamental para computação científica com python. <https://github.com/numpy/numpy>. Acesso em: junho de 2025.

Silva, R. S. C. (2025). social-graph-mining-github. <https://github.com/robinCrobin/social-graph-mining-github>. Acesso em: junho de 2025.